

Transport Jaan — Technical Documentation

Multi-vendor transport & logistics marketplace — platform architecture and module reference

Prepared 2026-08-03 · main_copy_dev branch

This is a working reference for the Transport Jaan platform as it exists in the codebase today — what's built, how the pieces fit together, and where the rough edges are. It's written for whoever is picking this project up next, whether that's a new developer joining the team or someone auditing scope before a handover.

1. What the platform does

Transport Jaan is a single Laravel application that runs several independent transport and logistics verticals under one roof: vehicle rental (land, air and sea), bus and train ticketing, courier/parcel delivery, warehouse storage, and a multimodal "chain several trips together" journey planner. Each vertical has its own booking flow, its own pricing logic, and in most cases its own set of database tables — there's very little shared abstraction between them by design (or by history; see section 9).

Three kinds of accounts use the system: clients booking services, vendors who list and fulfil those services (rental agencies, warehouse operators, courier companies), and a super admin who approves vendors, configures commissions, and has oversight across everything. Bus and train travel is the one exception — those schedules are run directly by the platform rather than by third-party vendors.

2. Stack

Layer	Choice
Backend	PHP 8.2, Laravel 12
Frontend	React 18 via Inertia.js 2 — no separate REST API for page loads, controllers hand props straight to React pages
Styling	Tailwind CSS 3
Build	Vite 6
Database	MySQL in production; <code>.env.example</code> defaults to SQLite for local dev
Search	Laravel Scout — driver is set to <code>collection</code> (in-memory) by default; <code>meilisearch/meilisearch-php</code> is pulled in as a dependency for when a real search index is wanted
Queue / Sessions / Cache	Database driver for all three — no Redis in use despite it being configured in <code>.env</code>
Auth	Laravel's own session-based auth. Sanctum is installed but not actively used for the web app
Payments	PayHere (Sri Lankan gateway) — currently wired into the Courier module only
Mail	SMTP via Zoho Mail
PDF/exports	barryvdh/laravel-dompdf server-side; jsPDF + html2canvas client-side for booking slips/quotes

3. Size and shape of the codebase

As of this write-up: 75 controllers, 101 Eloquent models, 150 migrations. `routes/web.php` is a single 2,400-line file — every route in the system lives there, with no split into per-domain route files. It works, but finding anything in it means searching, not browsing.

The Courier module is disproportionately large relative to the others — roughly 25 dedicated service classes under `app/Services/Courier` alone, covering things most of the other verticals don't have at all (see section 5.3). If you're estimating effort for a new feature, "how big is Courier" is not a good baseline for "how big is Warehouse" or "how big is Vehicle Rental."

4. Authentication and roles

Role is a plain string column on `users.role` — `client`, `vendor`, or `SuperAdmin`. There's no dedicated roles/permissions table for this top-level split; it's a simple column check, enforced through middleware like `RoleMiddleware` and `SuperAdminMiddleware`.

`spatie/laravel-permission` is installed, but it isn't used for that top-level role check — it backs a separate, much more granular RBAC system that exists only inside the Courier module (`CourierRoleProfile`, `app/Services/Rbac/CourierRoleModelService.php`, `ServiceWorkspaceManager`). That system supports per-team custom role profiles, temporary access grants with expiry, and a sensitive-action approval workflow. It's effectively a separate, smaller authorization system living inside the bigger one.

Vendors go through an approval gate before they can operate. `VendorProfile` carries a `submission_status` state machine — `draft` → `submitted` → `approved` / `rejected` / `revision_requested` — and the `VendorVerificationCheck` middleware blocks vendor-only routes until a vendor is approved. Client-side there's an equivalent `ClientVerificationCheck` middleware for the handful of routes that need it.

5. Module reference

5.1 Vehicle Rental (land / air / sea)

This is three parallel systems that happen to share a controller. Land rentals use `Booking`, `BookingSchedule`, `BookingAddon`, `BookingCustomer` and `BookingPayment`. Air rentals use the same shape of tables with an `AirVehicle` prefix. Sea rentals use a `SeaVehicle` prefix. There's no shared polymorphic booking table — it's the same logic copy-pasted three times with different model names, which means any fix to booking logic generally has to be applied in three places if it's meant to apply everywhere. `ClientBookingController.php` is over 1,500 lines because it's carrying quote/store/payment/confirm/cancel for all three vehicle types in one file.

On the vendor side, listing a vehicle goes through `AddUnit` / `EditUnit`, with type-specific attributes split into `LandVehicleSpec`, `AirVehicleSpec` and `SeaVehicleSpec` tables. `VehicleFeaturePricing` holds the vendor-configured extras (child seat, GPS, etc.) that show up as optional add-ons at checkout. `VehicleDocument`, `VehicleMedia`, `VehiclePolicy` and `VehicleMaintenance` round out a listing.

Two features worth calling out because they're easy to confuse with each other: vendors can assign one of their own drivers to a confirmed booking after the fact (`AssignDriverModal`, operational, land only so far) — that's separate from the client-facing self-drive/with-driver choice at booking time (`needs_driver` on `Booking`, land only, adds a percentage of the daily rate).

Cancellation refund percentages (how much you get back based on how close to pickup you cancel) are centralised in `VehicleBookingCancellationService` and shared across all three vehicle types, which is one of the few genuinely shared pieces of logic in this module.

5.2 Ticket Booking (Bus & Train)

Unlike vehicle rental, bus and train schedules aren't vendor listings — they're platform-owned data (`BusStation` / `BusSchedule` / `Bus`, and the equivalent `Train*` tables). Round trips are modelled as two linked one-way booking rows rather than a single row referencing two schedules — each leg gets its own `BusBooking` / `TrainBooking` record, tied together by a shared `round_trip_group_id` and a `leg` enum (`outbound` / `return`). That was a deliberate choice: it means every existing piece of single-leg logic — cancellation, expiry, ticket generation — keeps working unmodified for round trips, at the cost of two rows per round-trip booking instead of one.

Seat selection only exists for buses (a row/column grid built from `seatLayout`, with booked seats greyed out and, as of this pass, a male/female tag required per selected seat, stored as a `seat_genders` JSON map). Trains are booked by passenger count and class — there's no seat map for trains. Both flows show a route preview on Google Maps using station lat/lng pulled from the seeded station data.

5.3 Courier

This is the deepest module in the codebase, closer to a small internal SaaS product than a "book a courier" feature. It covers: shipment creation and tracking (`CourierShipment`, `CourierTrackingEvent`), COD (cash-on-delivery) settlement in batches with a reconciliation service, label generation and printing with configurable label sizes/templates, a PayHere payment integration

with signature verification on the notify webhook, and its own security layer — trusted devices, session security checks, temporary access grants that expire, a sensitive-action approval workflow, and a "break glass" emergency access path that fires alerts (email + optional webhook) when used. There's also an automated compliance/audit trail for COD integrity specifically (`CourierVendorCodIntegrityIncident` , `CourierCodIntegrityMonitoringService`).

If a client asks "can we get X security control on the rest of the platform," this module is the reference implementation — most of that pattern doesn't exist anywhere else yet.

5.4 Warehouse

Clients book storage space in a vendor's `WarehouseUnit` for a date range. Pricing is monthly rate × space utilisation × duration, plus a security deposit and setup fee, with VAT/tax currently zeroed out platform-wide (temporary, at the client's request — see appendix). As of this pass there's also an optional "Storage & Fulfilment" add-on (pick/pack/ship handling on the client's behalf) priced at 15% of the monthly rate.

The pricing calculation is duplicated across four separate frontend files (the search/quote widget, the checkout page, the payment page, and the booking-summary/confirmation page), each with its own copy of the same arithmetic. They've drifted from each other before and will again — this is the single biggest maintainability risk in the Warehouse module and is worth centralising into one server-side endpoint the frontend just calls.

5.5 Freight / Multimodal

`MultiModelJourney` , `MultiModelLeg` and `MultiModelBooking` let a client chain trips across different transport types into a single journey (e.g. bus leg then train leg). This is the least mature of the booking flows — it reuses the bus/train search UI inline rather than having its own, and the booking/payment side is thinner than the single-mode flows.

5.6 Super Admin

Platform oversight: approving/rejecting vendor submissions, configuring per-service-type commission percentages (`Commission` , calculated per booking into `CommissionEarning` via `CommissionCalculationService` when a payment lands — default split is 50/50 unless configured otherwise), payments oversight across modules, and the Courier-specific governance/audit tooling described above.

6. Notifications and email

In-app notifications live in a single `notifications` table shared across modules (with module-specific foreign keys bolted on as needed, e.g. `warehouse_booking_id`). Outbound email goes through Zoho's SMTP relay using a plain username/password (`MAIL_USERNAME` / `MAIL_PASSWORD` in `.env`). There's also a full set of Zoho OAuth credentials configured (`ZOHO_CLIENT_ID` / `SECRET` / `REFRESH_TOKEN`) that, as far as the current codebase goes, nothing actually calls — worth confirming with whoever set that up whether it's for a planned integration or leftover from something that got dropped.

7. Third-party integrations

Service	Used for
Google Maps JavaScript API	Route previews on bus/train booking, vehicle pickup/dropoff location, multimodal journey planning. One shared script-loading pattern is duplicated across a handful of map components rather than centralised.
PayHere	Payment gateway, currently wired into Courier shipment payments only — other modules take payment details (card/bank transfer/slip upload) without an actual gateway behind them yet.
Zoho Mail (SMTP)	Transactional email
Nominatim / ZipCodeBase / RestCountries / OpenDataSoft	Location and postal-code lookups, configured in <code>config/services.php</code> — usage is thin, worth checking exactly which flows call these before assuming they're load-bearing

8. Deployment notes

- Standard Laravel deploy: `composer install` , `npm run build` , `php artisan migrate` . `bootstrap/app.php` already has `trustProxies` configured for reverse-proxy setups.
- Do not run `php artisan serve` alongside a live Vite dev server behind a tunnel/proxy — the `public/hot` file Vite writes locks asset URLs to `localhost` , which breaks the page for anyone not on the same machine. Build the assets and serve the built output instead.
- Search defaults to Scout's in-memory `collection` driver, which does not scale past local/dev use — point `SCOUT_DRIVER` at Meilisearch before this goes to real production traffic with any meaningful data volume.

9. Known limitations, as of this write-up

- **Triplicated vehicle booking logic.** Land/air/sea vehicle bookings are three separate table sets and largely separate code paths inside one controller. A fix or feature applied to one doesn't apply to the others automatically.
- **Warehouse pricing duplicated across 4 frontend files.** See section 5.4.
- **One 2,400-line routes file.** No functional problem today, but it's the first thing that should be split if the routing layer starts feeling unmanageable.
- **PayHere only covers Courier.** Every other module's "payment" step records payment details without an actual gateway transaction behind it — fine for cash/bank-transfer/manual-slip flows, not fine if online card payment is expected to actually charge a card.
- **No seat map for trains**, unlike buses.
- **Multimodal/Freight is the least finished flow** relative to single-mode bus/train/vehicle booking.
- **VAT/tax is currently disabled on Warehouse bookings** at the client's request — re-enabling it is a one-line change per pricing file (see 5.4) once pricing is confirmed.

Also flagged separately: `.env.example` currently contains real, non-placeholder credentials (Zoho API secret/refresh token, an actual mail password, a live Google Maps key) committed to git history. This should be scrubbed to genuine placeholders and the exposed credentials rotated — it's not a documentation issue, it's an active leak, and is being handled as its own item outside this document.

This document reflects the codebase on the `main_copy_dev` branch as of the date above. It will drift out of date as the platform changes — treat it as a snapshot, not a live source of truth.